Files hidden in shared chats

Try to suggest python based tech stack for this and the first step for this

Python Tech Stack Suggestion

Based on the flowchart, this is a **two-sided admin portal** (DC side + CHIPS Admin side) with registration workflows, notifications, and dashboards.

Recommended Stack

Backend

- **FastAPI** — async REST APIs, auto-generated docs (perfect for request/response flows shown)
- **SQLAlchemy** + **Alembic** — ORM for managing station IDs, L1/L2 request states, migrations
- **PostgreSQL** — relational DB to track DCs, requests, statuses, and mappings
- **Celery** + **Redis** — background task queue for sending notifications (L1 done, L2 done, station ID assigned)

Frontend (Python-friendly options)

- **Streamlit** — fastest way to build the dashboards (DC dashboard + CHIPS dashboard)
- *Or* **React** + **FastAPI** if you need a more polished UI

Auth & Notifications

- **FastAPI-Users** — role-based auth (DC role vs CHIPS Admin role)
- **SMTP / Twilio** — email/SMS notifications on status changes

Excel Export

- **openpyxl / pandas** — for the "export Excel" features visible in the flow

---

First Step: Database Schema Design

This is the most critical foundation. You need to model:

python

```python
# models.py (SQLAlchemy)

class District(Base):
    __tablename__ = "districts"
    id          = Column(Integer, primary_key=True)
    dc_name     = Column(String, nullable=False)
    district    = Column(String, nullable=False)
    station_id  = Column(String, unique=True, nullable=True)  # assigned by CH
    created_at  = Column(DateTime, default=func.now())

class StationRequest(Base):
    __tablename__ = "station_requests"
    id          = Column(Integer, primary_key=True)
    district_id = Column(ForeignKey("districts.id"))
    status      = Column(Enum("pending", "done"), default="pending")
    requested_at = Column(DateTime, default=func.now())

class L1Request(Base):
    __tablename__ = "l1_requests"
    id            = Column(Integer, primary_key=True)
    district_id   = Column(ForeignKey("districts.id"))
    station_id    = Column(String, nullable=False)
    machine_id    = Column(String, nullable=False)
    operator_name = Column(String, nullable=False)
    uv_id         = Column(String, nullable=False)
    status        = Column(Enum("pending", "done"), default="pending")

class L2Request(Base):
    __tablename__ = "l2_requests"
    id          = Column(Integer, primary_key=True)
    district_id = Column(ForeignKey("districts.id"))
    # L2-specific fields
    status      = Column(Enum("pending", "done"), default="pending")
```

Why start here? Every API route, every dashboard widget, and every notification trigger depends on this schema. Getting it right first prevents painful migrations later.

Immediate Action Items

1. Set up a **PostgreSQL DB** locally (via Docker: `docker run -e POSTGRES_PASSWORD=pass -p 5432:5432 postgres`)
2. Create a **FastAPI project** with `fastapi`, `sqlalchemy`, `asyncpg`, `alembic`
3. Write and run the **initial Alembic migration** with the schema above
4. Build the **two auth roles**: DC and CHIPS Admin

Want me to scaffold the full FastAPI project structure or write the Alembic migration files?

what is sql alchemy?

SQLAlchemy

SQLAlchemy is a Python library that lets you **interact with a database using Python code instead of raw SQL queries.**

The Problem it Solves

Without SQLAlchemy, you'd write raw SQL like this:

```python
cursor.execute("SELECT * FROM l1_requests WHERE status = 'pending'")
rows = cursor.fetchall()
```

With SQLAlchemy, you write Python instead:

```python
requests = session.query(L1Request).filter(L1Request.status == "pending").all(
```

Both do the same thing — but the Python version is cleaner, safer, and easier to maintain.

Two Main Parts

1. ORM (Object Relational Mapper) Maps your Python classes → database tables. Each class = one table, each object = one row.

```python
# This Python class...
class L1Request(Base):
    __tablename__ = "l1_requests"
    id     = Column(Integer, primary_key=True)
    status = Column(String)

# ...automatically creates this SQL table:
# CREATE TABLE l1_requests (id SERIAL PRIMARY KEY, status VARCHAR);
```

2. Core (Query Engine) For writing more complex queries when you need fine-grained SQL control, without giving up Python syntax.

Why Use it in Your CHIPS Project

Your Need	SQLAlchemy Handles
Store DC requests	Insert rows into tables
CHIPS Admin marks L1 done	Update `status` column
DC dashboard	Query requests by district
Export to Excel	Fetch rows → pandas → `.xlsx`

Simple Analogy

Think of SQLAlchemy as a **translator** — you speak Python, your database speaks SQL, and SQLAlchemy sits in the middle translating both ways automatically.

Want me to show a working example with your specific CHIPS workflow?

> Give me the setup required for this , tech stack that need to be installed and how can we divide projects to three members

Project Setup & Team Division

Tech Stack Installation

1. Prerequisites

```bash
# Install Python 3.11+
python --version

# Install PostgreSQL
# Download from https://www.postgresql.org/download/

# Install Redis (for notifications)
# Windows: https://redis.io/download
# Mac:
brew install redis
```

2. Create Project & Virtual Environment

```bash
mkdir chips-portal
cd chips-portal
python -m venv venv

# Activate
venv\Scripts\activate        # Windows
source venv/bin/activate     # Mac/Linux
```

3. Install All Required Packages

bash

```
pip install fastapi                    # Web framework
pip install uvicorn                    # Server to run FastAPI
pip install sqlalchemy                 # ORM (database layer)
pip install alembic                    # Database migrations
pip install asyncpg                    # PostgreSQL async driver
pip install psycopg2-binary            # PostgreSQL sync driver
pip install python-jose[cryptography] # JWT auth tokens
pip install passlib[bcrypt]            # Password hashing
pip install python-multipart           # Form data handling
pip install celery                     # Background tasks
pip install redis                      # Redis client
pip install pandas                     # Data manipulation
pip install openpyxl                   # Excel export
pip install streamlit                  # Dashboard UI
pip install python-dotenv              # Environment variables
pip install pydantic[email]            # Data validation
```

4. Save dependencies

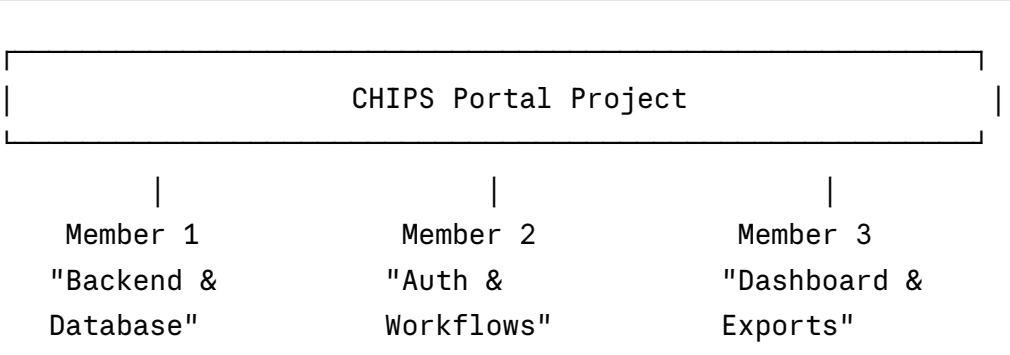
bash

```
pip freeze > requirements.txt
```

Project Folder Structure

```
chips-portal/
│
├── backend/
│   ├── main.py                  # FastAPI app entry point
│   ├── database.py              # DB connection setup
│   ├── models/
│   │   ├── district.py          # District & StationRequest models
│   │   ├── l1_request.py        # L1Request model
│   │   └── l2_request.py        # L2Request model
│   ├── routers/
│   │   ├── auth.py              # Login, token, roles
│   │   ├── station.py           # Station ID request/assign
│   │   ├── l1.py                # L1 registration routes
│   │   └── l2.py                # L2 registration routes
│   ├── schemas/                 # Pydantic input/output shapes
│   ├── services/                # Business logic
│   └── tasks.py                 # Celery notification tasks
│
├── dashboard/
│   ├── dc_dashboard.py          # Streamlit DC view
│   └── chips_dashboard.py       # Streamlit CHIPS Admin view
│
├── alembic/                     # DB migrations
├── .env                         # Secrets & DB URL
├── requirements.txt
└── README.md
```

Team Division (3 Members)



👤 Member 1 — Database & Core Backend

Owns: `models/`, `database.py`, `alembic/`, `routers/station.py`

```
Responsibilities:
✅ Design & create all SQLAlchemy models
✅ Set up PostgreSQL connection
✅ Write & run Alembic migrations
✅ Build Station ID request & assignment APIs
✅ Write base CRUD functions others will use
```

First task:

```python
# database.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base
from dotenv import load_dotenv
import os

load_dotenv()
DATABASE_URL = os.getenv("DATABASE_URL")

engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(bind=engine)
Base = declarative_base()
```

👤 Member 2 — Auth & L1/L2 Registration Workflows

Owns: `routers/auth.py`, `routers/l1.py`, `routers/l2.py`, `services/`, `tasks.py`

```
Responsibilities:
✅ Implement JWT login for DC & CHIPS Admin roles
✅ Build L1 registration submit & review APIs
✅ Build L2 registration submit & review APIs
✅ Mark L1/L2 as done + trigger notifications
✅ Set up Celery + Redis for background notifications
```

First task:

python

```python
# routers/auth.py
from fastapi import APIRouter, Depends
from fastapi.security import OAuth2PasswordRequestForm

router = APIRouter()

@router.post("/login")
def login(form: OAuth2PasswordRequestForm = Depends()):
    # validate user, return JWT token
    pass
```

👤 Member 3 — Dashboards & Excel Export

Owns: `dashboard/dc_dashboard.py`, `dashboard/chips_dashboard.py`, `services/export.py`

```
Responsibilities:
✅ Build DC Dashboard (track own requests & statuses)
✅ Build CHIPS Admin Dashboard (overview of all DCs)
✅ Excel export for Station IDs, L1, L2 lists
✅ Connect Streamlit UI to backend APIs
✅ Display live status updates
```

First task:

python

```python
# dashboard/dc_dashboard.py
import streamlit as st
import requests

st.title("DC Dashboard")

response = requests.get("http://localhost:8000/station/my-requests")
data = response.json()

st.dataframe(data)   # Show requests table
```

`.env` File (shared by all)

env

```
DATABASE_URL=postgresql://postgres:password@localhost:5432/chipsdb
SECRET_KEY=your-secret-key-here
REDIS_URL=redis://localhost:6379
```

Week 1 Goal for All Three

Member	Day 1-2	Day 3-5
M1	Set up DB, write models	Run migrations, test connection
M2	Set up FastAPI app, auth routes	JWT login working for both roles
M3	Set up Streamlit, connect to API	Basic dashboard showing mock data

---

Want me to write the complete starter code for any specific member's part?

Can you explain the workflow for developing this

Development Workflow

Big Picture — How All Pieces Connect

```
[Streamlit UI]  ↔  [FastAPI Backend]  ↔  [PostgreSQL DB]
                          ↕
                   [Redis + Celery]
                  (Notifications)
```

Phase-by-Phase Development Workflow

Phase 1 — Foundation (Week 1)

Member 1 leads, others follow

Step 1: Database First

This is the absolute starting point. Everything else depends on this.

```
DC submits request
       ↓
Where does it get saved? → PostgreSQL Table
What does the table look like? → SQLAlchemy Model
How does the table get created? → Alembic Migration
```

The flow is always:

```
Write Model → Generate Migration → Run Migration → Table Created in DB
```

python

```python
# Step 1: Write the model (Member 1)
class L1Request(Base):
    __tablename__ = "l1_requests"
    id            = Column(Integer, primary_key=True)
    station_id    = Column(String, nullable=False)
    machine_id    = Column(String, nullable=False)
    operator_name = Column(String, nullable=False)
    status        = Column(String, default="pending")

# Step 2: Generate migration
# alembic revision --autogenerate -m "create l1_requests table"

# Step 3: Run migration
# alembic upgrade head
```

Step 2: FastAPI App Skeleton

Once DB is ready, Member 2 sets up the API skeleton:

python

```python
# main.py
from fastapi import FastAPI
from routers import auth, station, l1, l2

app = FastAPI(title="CHIPS Portal")

# Register all route groups
app.include_router(auth.router,    prefix="/auth")
app.include_router(station.router, prefix="/station")
app.include_router(l1.router,      prefix="/l1")
app.include_router(l2.router,      prefix="/l2")

# Run with: uvicorn main:app --reload
```

Step 3: Auth System

Before ANY feature is built, login must work — because every API route checks who is calling it.

```
User visits app
      ↓
Enters username + password
      ↓
FastAPI checks DB → is this a real user?
      ↓
Yes → Generate JWT Token → send back to user
      ↓
User sends token with every future request
      ↓
FastAPI checks token → is this a DC or CHIPS Admin?
      ↓
Allow or Deny access to that route
```

Two roles in your system:

python

```python
# DC can only:   submit requests, view own status
# CHIPS Admin:   view all requests, mark as done, assign IDs

class UserRole(str, Enum):
    DC          = "dc"
    CHIPS_ADMIN = "chips_admin"
```

Phase 2 — Core Features (Week 2)

All 3 members work in parallel

Here's how the workflow maps to actual API calls:

Workflow 1 — Station ID Flow

```
DC fills form → POST /station/request
                        ↓
              DB saves request (status=pending)
                        ↓
CHIPS Admin opens tab → GET /station/all-requests
                        ↓
              Sees list of pending requests
                        ↓
Assigns ID → PUT /station/assign/{request_id}
                        ↓
              DB updates: station_id assigned, status=done
                        ↓
DC gets notified → Celery sends email/SMS
```

python

```python
# routers/station.py

# DC submits request
@router.post("/request")
def request_station_id(data: StationRequestSchema, db: Session = Depends(get_d
    new_request = StationRequest(district_id=data.district_id, status="pending
    db.add(new_request)
    db.commit()
    return {"message": "Request submitted"}

# CHIPS Admin assigns ID
@router.put("/assign/{request_id}")
def assign_station_id(request_id: int, station_id: str, db: Session = Depends(
    req = db.query(StationRequest).filter_by(id=request_id).first()
    req.station_id = station_id
    req.status = "done"
    db.commit()
    notify_dc.delay(req.district_id)   # Celery background task
    return {"message": "Station ID assigned"}
```

Workflow 2 — L1 Registration Flow

```
DC submits L1 form → POST /l1/submit
    (station_id, machine_id, operator_name, uv_id, password)
                          ↓
              DB saves L1Request (status=pending)
                          ↓
CHIPS Admin → GET /l1/all-requests  (sees full list)
                          ↓
Reviews & processes → PUT /l1/mark-done/{id}
                          ↓
              DB updates status=done
                          ↓
DC notified → "L1 Complete"
```

Workflow 3 — L2 Registration Flow

Exact same pattern as L1, just different data fields

```
POST /l2/submit  →  GET /l2/all-requests  →  PUT /l2/mark-done/{id}
```

Phase 3 — Notifications (Week 2-3)

Member 2 handles this

Celery runs tasks **in the background** so the API doesn't slow down:

```
API receives request
       ↓
Saves to DB (instant)
       ↓
Sends task to Redis queue (instant)
       ↓
API responds to user immediately ✅
       ↓              ↓
              Celery worker picks up task
                       ↓
              Sends email/SMS in background
```

python

```python
# tasks.py
from celery import Celery

celery_app = Celery("chips", broker="redis://localhost:6379")

@celery_app.task
def notify_dc(district_id: int):
    # fetch DC email from DB
    # send email: "Your Station ID has been assigned"
    pass

@celery_app.task
def notify_l1_done(district_id: int):
    # send email: "L1 registration complete"
    pass
```

Phase 4 — Dashboards (Week 3)

Member 3 leads

DC Dashboard flow:

```
DC logs in
    ↓
Streamlit calls GET /station/my-requests
Streamlit calls GET /l1/my-status
Streamlit calls GET /l2/my-status
    ↓
Display all statuses in one view
    ↓
DC sees: ✅ Station ID received | ⏳ L1 Pending | 🔒 L2 Locked
```

CHIPS Admin Dashboard flow:

```
Admin logs in
    ↓
GET /station/all-requests → show pending count
GET /l1/all-requests      → show L1 list
GET /l2/all-requests      → show L2 list
    ↓
Export button → GET /export/l1-excel → downloads .xlsx
```

python

```python
# dashboard/dc_dashboard.py
import streamlit as st
import requests

token = st.session_state.get("token")
headers = {"Authorization": f"Bearer {token}"}

col1, col2, col3 = st.columns(3)

with col1:
    st.metric("Station ID", "✅ Received")
with col2:
    st.metric("L1 Status", "⏳ Pending")
with col3:
    st.metric("L2 Status", "🔒 Not Started")
```

Phase 5 — Excel Export (Week 3)

Member 3 handles

python

```python
# services/export.py
import pandas as pd
from fastapi.responses import StreamingResponse
import io

def export_l1_to_excel(db: Session):
    requests = db.query(L1Request).all()

    df = pd.DataFrame([{
        "Station ID":    r.station_id,
        "Machine ID":    r.machine_id,
        "Operator Name": r.operator_name,
        "Status":        r.status
    } for r in requests])

    buffer = io.BytesIO()
    df.to_excel(buffer, index=False)
    buffer.seek(0)

    return StreamingResponse(buffer,
        media_type="application/vnd.ms-excel",
        headers={"Content-Disposition": "attachment; filename=l1_requests.xlsx
    )
```

Overall Development Timeline

```
Week 1          Week 2              Week 3
─────────────   ─────────────────   ──────────────────
DB Models    →  Station ID APIs  →  DC Dashboard
Migrations   →  L1/L2 APIs       →  CHIPS Dashboard
Auth/Login   →  Notifications    →  Excel Export
                                 →  Testing & Polish
```

Daily Git Workflow (All Members)

bash

```bash
# Start of day
git pull origin main

# Work on your feature
git checkout -b feature/l1-api      # create your branch

# End of day
git add .
git commit -m "feat: add L1 submit endpoint"
git push origin feature/l1-api

# When feature is done → raise Pull Request → others review → merge to main
```

Want me to write the complete starter code for any specific phase or member?